

태양광 발전량 예측 B2B 서비스 기획안

Oct 1, 2026 · @이민기

배경 및 문제 정의

태양광은 날씨에 따라 발전량이 널뛰기 때문에, 발전량을 모르면 전력을 팔 수도 운영할 수도 없습니다. 실습에서 만든 HAIC 주가 예측 파이프라인(20일 시퀀스 → 다음 날 증가)은 구조를 거의 그대로 두고 "다음 날 시간별 발전량" 예측으로 바꿀 수 있습니다.

- 한국은 2021년부터 전력거래소 "재생에너지 발전량 예측제도"를 운영합니다. 하루 전 예측 오차가 작은 사업자에게 정산금을 주고, 오차가 크면 받지 못합니다 (오차 6% 이하 kWh당 약 4원, 6~8% 약 3원 — 수치는 대략치, 확인 필요).
- 제주·호남처럼 태양광이 몰린 지역은 공급 과잉 시 출력제어(발전 중단 명령)를 받습니다. 예측이 틀리면 수익이 사라지는 날이 생깁니다.
- 발전소 한 곳의 발전량은 작아서, 여러 발전소를 모아 20MW 이상으로 묶는 "집합자원" 사업자(VPP·전력중개사업자)가 예측의 실질 주체입니다.

이 과제는 중개사업자와 발전소 운영사가 겪는 "내일 얼마나 발전할지 모른다"는 문제를, 실습 과정에서 익힌 **학습 → 서빙 → 드리프트 감지 → 자동 재학습** 루프로 푸는 B2B 예측 서비스 기획입니다.

타깃 고객과 페인포인트 (기획서 ①)

1차 고객은 여러 태양광 발전소를 모아 전력시장에 예측량을 제출하는 **전력중개사업자(VPP)**입니다. 예측 오차가 곱 정산금 손실로 이어지는 유일한 주체라 돈을 낼 이유가 가장 분명합니다.

이해관계자	지금 겪는 불편 (Pain Point)	예측 서비스가 고장 나면 생기는 일
전력중개사업자 (VPP) 운영 담당	매일 오전 10시 전에 다음 날 시간별 예측량을 제출해야 하는데, 엑셀과 기상청 예보로 수작업 → 오차 8% 초과 시 정산금 0원	제출 마감 놓침 = 그날 정산금 전액 사라짐
발전소 소유주(발전사업자)	내 발전소가 정상인지 판단할 기준이 없음. 인버터 고장·패널 오염으로 발전량이 줄어도 몇 주 뒤 정산서를 보고 알게 됨	예측값이 틀린 채 계속 나가면 고장을 "날씨 탓"으로 오해
중개사업자 데이터·모델 담당	계절이 바뀌거나 새 발전소가 추가될 때마다 모델이 조용히 틀리기 시작하는데, 누가 알려주지 않음	재학습 시점을 놓쳐 수주 동안 정산금 손실 누적

이해관계자	지금 겪는 불편 (Pain Point)	예측 서비스가 고장 나면 생기는 일
중개사업자 경영진	발전소를 늘릴수록 예측 인력도 늘려야 해서 마진이 안 남음	예측 품질이 얼마나 나빠지는지 보고 받지 못해 고객(발전소) 이탈

제출 마감 시간과 정산 기준은 전력거래소 규정을 팀에서 한 번 확인해서 기획서에 인용하는 게 좋습니다.

서비스 컨셉과 운영 목표 (기획서 ②)

"SolarCast" (가칭) — 발전소별 시간당 발전량 예측 API. 중개사업자가 발전소의 최근 실적과 날씨 예보를 보내면 다음 날 발전량을 돌려주고, 예측이 틀리기 시작하면 스스로 재학습합니다.

핵심 기능 (실습 파이프라인 1:1 대응)

기능	해결하는 Pain Point	HAIC 실습에서 올 것
발전량 예측 API (<code>/predict</code>)	수작업 예측 → API 호출 한 번	<code>/predict</code> + Lazy/Eager 로딩
발전 실적 업로드 (<code>/data/upload</code>)	발전소 추가·실적 갱신을 CSV 한 장으로	<code>/data/upload</code> (완성형 그대로)
배포 게이트	나빠진 모델이 운영에 나가는 사고 차단	MLflow RMSE 게이트 → Production 승격
드리프트 감시 + 자동 재학습	"모델이 조용히 틀리기 시작"을 당일 감지	<code>batch-test</code> → <code>drift_detector</code> → <code>fine_tune</code>
운영 로그 조회 (<code>/logs</code>)	경영진·고객에게 "언제 왜 재학습했는지" 증거 제공	<code>aiops.log</code> 패널

운영 목표 (발표 시 숫자로 말할 것)

- 품질: 일 발전량 기준 오차율 **8% 이하** (정산금 기준선과 맞춤). 지표는 nRMSE(설비용량 대비 RMSE) 또는 MAPE 중 하나로 팀이 고정.
- 응답 시간: `/predict` 1초 이내. 제출 마감 전 수백 개 발전소를 연속 호출해야 하므로 Eager 로딩 채택.
- 가용성: 매일 제출 마감 전 1시간은 무조건 응답 (재학습 중에도 기존 Production 버전이 계속 응답).
- 드리프트 대응: 감지 후 당일 내 재학습·게이트 재검증·재배포 완료.

데이터·모델·운영 설계 (기획서 ③, 도메인 매핑표)

실습 가이드 7번 도메인 매핑표를 태양광으로 채운 결과입니다. 이 표가 정해지면 스키마·게이트·임계값·API가 따라옵니다.

템플릿 개념 (HAIC)	SolarCast
예측 대상: 다음 날 종가	다음 날 발전소별 일 발전량(kWh) — 1차 목표. 시간별 24개 예측은 확장 과제
입력 시퀀스: 최근 20거래일 (close, volume)	최근 14일 (일 발전량, 일사량, 평균기온, 운량) + 다음 날 기상 예보 3개
데이터 공급: 대시보드 CSV 업로드	전날 발전 실적 + 기상 관측 CSV 업로드 (컬럼: Date, Generation, Irradiance, Temp, Cloud)
배포 게이트: RMSE ≤ \$4.00	nRMSE ≤ 8% (설비용량 대비) — 정산금 기준과 동일한 선
드리프트 신호: 변동성 3배 급변	계절 전환(장마·폭설), 패널 오염·열화, 인버터 교체, 신규 발전소 편입
재학습: 최근 21거래일 fine-tuning	최근 30일 데이터로 Production 가중치에서 warm-start fine-tuning (10 epoch)
알림: aiops.log [WARN]	aiops.log + 중개사업자 운영 담당 메신저 알림 (데모는 로그 패널로 대체)
이해관계자: 투자 분석가, 운영자	VPP 운영 담당(정산금), 발전소 소유주(고장 조기 발견), 모델 담당(재학습), 경영진(마진)

예측 단위 제안: 모델은 일 발전량, 제출 포맷은 시간별 24개

결론: 실제 제도는 시간별 제출을 요구하지만, 과제 범위에서는 "일 발전량 LSTM + 시간 프로파일 분해"로 24개 값을 만드는 방식을 권합니다. 모델·게이트·드리프트 루프는 그대로 두고 후처리 함수 하나만 추가하면 됩니다.

구분	실제 제도(재생에너지 발전량 예측제도)	과제에서 할 수 있는 범위
제출 단위	다음 날 1시간 단위 24개 (전날 10시 제출, 17시 수정)	일 발전량 1개 → 시간 프로파일로 24개 분해
평가 기준	시간별 오차율 = 예측-실제 ÷ 설비용량, 발전량이 설비용량 10% 이상인 시간대만	일 nRMSE(설비용량×24 대비)로 게이트, 분해 후 시간별 오차율은 데모에서 1회 계산
정산	오차율 8% 이하 3원/kWh, 6% 이하 4원/kWh	8%를 배포 게이트·드리프트 임계값으로 사용
대상	20MW 이상 집합자원(소규모 발전소 모집 포함)	삼천포 2호기 1개소로 검증, 집합은 합산으로 설명

단계별 구현 범위

1. 1단계(과제 필수): 최근 14일 일 발전량 → 다음 날 일 발전량. 실습 코드 수정량이 가장 적고 3일 안에 루프 전체를 돌릴 수 있음.
2. 2단계(과제 안에서 가능): 최근 30일의 같은 시간대 발전 비율 평균을 프로파일로 저장하고, 일 예측 값에 곱해 24개 시간값 산출. 원본 월별 CSV에 1~24시 발전량이 이미 있어 추가 수집 없음.
/predict 응답에 hourly_kwh 배열을 더하면 Swagger 캡처도 제도 포맷과 맞음.
3. 3단계(확장 과제): 시간별 LSTM 직접 학습(입력 7일×24시간). 기상 시간별 데이터가 1년치뿐이라 학습 기간이 짧고, 검증 시간이 부족해 발표에서는 로드맵으로만 제시.

시간 프로파일 분해란 "하루 총량을 시간대별로 나누는 비율표"를 과거 실적에서 만들어, 일 예측값에 곱하는 것입니다. 최근 30일 각 날의 (시간 발전량 ÷ 그날 총량)을 시간별로 평균 내면 합이 1인 비율 24개가 나오고, 내일 예측 총량이 4,000kWh이면 13시 비율 14%×4,000 = 560kWh가 13시 예측값이 됩니다. 필요한 것은 원본 월별 CSV의 1~24시 컬럼뿐이라 추가 수집 없이 함수 하나로 구현됩니다.

삼천포 2호기 실적(2025-09~2026-08, 설비용량 1MW 가정, 피크 847kWh/h)으로 검증한 분해 자체의 오차입니다. 일 총량을 정확히 안다고 가정하고, 제도 방식대로 발전량이 설비용량 10% 이상인 시간대만 평가했습니다.

방식	시간별 오차율 평균	8% 이하인 날 비율
최근 30일 프로파일 분해	6.0% (중앙값 5.2%)	77%
전날 같은 시간 그대로 사용	14.1%	—

즉 일 총량만 맞으면 분해만으로도 정산 기준 근처에 들어오고, 남는 오차는 날씨가 오전·오후 한쪽에만 흐린 날에서 나옵니다. 이 부분은 기상예보를 프로파일 보정에 쓰면 줄일 수 있습니다.

기상예보 데이터는 무료로 받을 수 있습니다.

출처	제공 항목	비고
기상청 단기예보 API (공공 데이터포털)	3일, 1시간 단위 기온·하늘상태·강수확률·습도	무료 인증키, 일사량 예보는 없음
Open-Meteo	1시간 단위 일사량·기온·운량 예보, 2022년 이후 과거 예보 아카이브	키 없이 URL 호출, 학습용 과거 예보도 확보 가능

과제에서는 Open-Meteo 하나로 학습(과거 예보)과 서빙(내일 예보)을 같은 형식으로 맞출 수 있어 가장 단순합니다. 1단계는 기상 없이 발전량만으로 돌리고, 기상 피쳐는 일 모델의 입력에 더하는 2단계 개선 항목으로 둡니다.

발표에서는 "제도는 시간별을 요구한다 → 우리는 일 모델을 먼저 검증하고 분해로 포맷을 맞춘다 → 데이터가 쌓이면 시간별 모델로 교체"로 정리하면 제도 이해와 구현 현실성을 동시에 보여줄 수 있습니다.

데이터 (이미 확보한 것 — team_solar/ 폴더)

- 발전 실적: 한국남동발전 공개 데이터 — 삼천포 태양광 2호기 일 발전량 2023-01-01 ~ 2026-08-31 (1,339일, 시간별 24컴럼 원본도 월별 CSV로 보관). 정제본 `data/samcheonpo2_daily.csv` : Date, generation_kwh, max_hourly_kwh, quality (ok 1,320일 / missing_suspect 19일 — 학습에서 제외).
- 기상: 기상청 ASOS 시간 관측 여수·순천·광양 3개 지점, 2025-09-30 ~ 2026-09-30 (1년). 기온·일조·일사·전운량 포함.
- 맞추어야 할 점: 발전 실적은 3년 8개월인데 기상은 최근 1년뿐. 1차는 발전량 단일 피쳐(+ 요일·계절 인코딩)로 전체 기간을 학습하고, 기상은 ASOS에서 2023~2025 추가 다운로드한 뒤 피쳐로 붙인다 (담당: 기획·데이터).
- 그래프에서 보이는 드리프트 후보: 매년 6/20~7/31 장마철에 30일 평균이 4 MWh → 2 MWh로 반 토막 나고, 2023년 9~10월에 발전량 0 기록일(연휴·정비 추정)이 몰려 있다. 드리프트 데모는 이 실제 구간을 주입하면 된다.

모델: 실습과 같은 3층 LSTM을 그대로 쓰되, 입력 피쳐 수만 2 → 4~7로 늘립니다 (`lstm_model.py` 의 `N_FEATURES` , `data/features.py` 의 스케일러를 컴럼별 min-max로 확장). 시간이 남으면 "전날 값 그대로" 베이스라인과 비교해 LSTM이 정말 나은지 보여주면 설득력이 커집니다.

운영 정책 (임계값 근거까지)

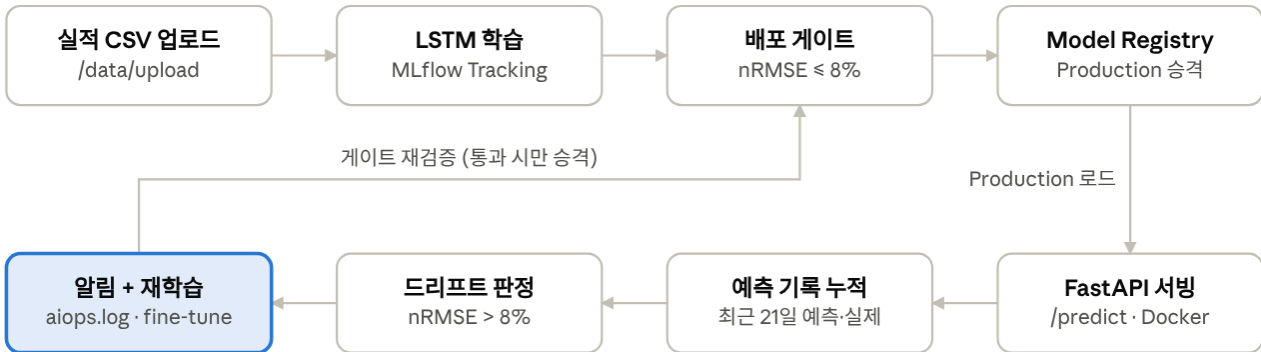
1. 입력 검증: 시퀀스 길이 14 고정, 발전량 ≥ 0 , 일사량 ≥ 0 , 운량 0~10 — 벗어나면 422.
2. 배포 게이트: 검증셋 nRMSE $\leq 8\%$. 통과 못 하면 등록만 하고 Production 유지.
3. 드리프트 판정: 최근 21일 (predicted, actual) nRMSE $> 8\%$ (실습의 WINDOW_SIZE·임계값 구조 그대로). 21일 = 지나가는 비 한번(2~3일)에 오탐하지 않을 길이.
4. 재학습: 최근 30일 + 선행 14일 = 44행으로 fine-tuning → 게이트 재검증 → 통과 시만 승격.
5. 가이드 Q&A가 지정한 약점 보완: 재학습 검증셋이 5개 남짓으로 너무 작으므로, 최근 30일은 학습하고 게이트는 직전 21일 드리프트 윈도우로 평가. 이 한 줄이 가산점 포인트.

시스템 아키텍처와 API 명세 (기획서 ④·⑤)

실습 `serving_app/` 구조를 그대로 쓰고 이름·피쳐·임계값만 바꿉니다. 새로 만드는 코드는 거의 없고, 팀이 손대는 파일은 `data/features.py` (피쳐·스케일러), `schemas.py` (입력 검증), `drift_detector.py` (지표·임계값), `simulate_drift.py` (드리프트 시나리오) 네 개입니다.

예측이 틀리면 스스로 재학습해 게이트를 다시 통과한다

MLOps — 학습 · 게이트 · 등록 (왼쪽에서 오른쪽)



AIOps — 서빙 · 감시 · 자동 재학습 (오른쪽에서 왼쪽, 다시 게이트로)

SolarCast 파이프라인 · 학습→게이트→서빙→감시→재학습 루프

위 줄은 Day2 MLOps, 아래 줄은 Day3 AIOps입니다. 재학습된 모델은 반드시 게이트를 다시 거치고, 탈락하면 기존 Production이 그대로 응답합니다.

API 명세 (Swagger /docs 에서 캐처해 발표 자료에 붙일 것)

Method	URL	역할	요청 예	응답 예 / 에러
POST	/predict	다음 날 발전량 예측	<code>{"plant_id":"P01","sequence":[{"generation":1820,"irradiance":4.2,"temp":21.3,"cloud":3}, ... 14개]}</code>	<code>{"predicted_kwh":1760.5,"model_version":"production"} / 422: 길이#14, 음수</code>
GET	/health	생존·모델 상태	—	<code>{"status":"ok","model_loaded":true,"loading_mode":"eager"}</code>
POST	/data/upload	발전 실적 CSV 업로드	multipart file (Date, Generation, Irradiance, Temp, Cloud)	<code>{"filename":"solar_....csv","rows":730} / 400: 컬럼 누락, 행 부족</code>
GET	/data/status	최신 업로드 요약	—	<code>{"exists":true,"rows":730,"start_date":...}</code>
POST	/predict/batch-test	드리프트 시뮬레이션 주입	<code>{"values":[... 35개 일 발전량]}</code>	<code>{"predictions":[...],"drift_check":{"status":"retrain_triggered","promoted":true,"nrmse":0.06}}</code>
GET	/logs, /logs/{file}	재학습 로그 조회	—	<code>[WARN] drift detected → [INFO] retrain triggered → [OK] new_nrmse=...</code>

진행 계획과 역할 분담

개인 실습 코드가 끝까지 돌아가는 팀원의 `project/` 를 기준 코드로 잡고, 그 위에 네 명이 동시에 붙입니다. 가이드가 말하는 대로 결과물은 하나의 파이프라인이어야 하므로, GitHub 저장소 하나에 모두 커밋합니다.

1. 도메인 매핑표 확정 + 데이터 선택 (팀 전체, 30분) — 위 4번 표의 빈칸·물음표를 없앤다.
2. CSV 정제해 Date, Generation, Irradiance, Temp, Cloud 형식으로 만들고 /data/upload 로 올린다.
3. data/features.py 피쳐 4개로 확장, schemas.py 입력 검증 수정, lstm_model.py N_FEATURES 변경 → baseline 학습 → /predict 응답 확인.
4. train_and_register.py 게이트를 nRMSE 8%로 바꾸고 Production 승격 → MODEL_SOURCE=mlflow 기동 → Docker 빌드.
5. simulate_drift.py 를 "장마 시나리오"(일사량 급감, 발전량 절반)로 바꿔 주입 → aiops.log 3줄 → 재배포 후 /predict 버전 변경 확인.
6. 스냅샷 6종 캡처(기획서 ⑥) → 발표 자료에 ①~⑥ 순서로 배치 → 리허설.

역할	담당	만드는 것	손대는 파일
기획·발표		페인포인트·운영 목표·운영 설계 (①~③), 발표 자료, 데이터 선별	이 문서, CSV
서빙 API		피쳐 확장, 입력 검증(422), /predict 응답, Lazy/Eager 측정 표	data/features.py, schemas.py, model_loader.py
학습·배포		nRMSE 게이트, MLflow 등록·승격, Docker 빌드·실행 로그	train_and_register.py, lstm_model.py, Dockerfile
모니터링·재학습		드리프트 시나리오, 임계값 근거, aiops.log 증거, 재학습 검증셋 보완	drift_detector.py, retrain_trigger.py, simulate_drift.py

담당 칸은 팀원 이름으로 채워 넣으세요. 3명이면 기획·발표를 서빙 API 담당이 겸합니다.

구현 결과와 데모 화면 (2026-10-01 클라우드 실행)

위 계획 2~6번을 project_solar/ 에서 끝까지 돌렸습니다. 모델은 발전량 14일 단일 피쳐 LSTM(계절 피쳐는 도움 안 됨, 기상은 2단계), 게이트·드리프트 임계값은 nRMSE 8%(설비용량 1MW 가정 × 24h 대비)입니다.

단계	결과
Day1 baseline	RMSE 1,386 kWh, nRMSE 5.78% (테스트 265일)
Day2 게이트	[GATE PASSED] nrmse=5.78% <= 8% → SolarCast_Predictor v1 Production

단계	결과
Day3 정상 배치 (최근 35일)	nRMSE 4.43% → ok
Day3 장마철 배치 (2026-06-20~)	nRMSE 4.19% → ok (계절 변동은 학습 분포 안)
Day3 신규 발전소 편입 (×1.8)	nRMSE 10.02% → 드리프트 → fine-tune 4.75% → v2 재배포
재배포 후 /predict	model_version: production-v1 → production-v2 (같은 요청)

```
[GATE PASSED] nrmse=5.78% <= 8% -> SolarCast_Predictor v1 promoted to
Production
03:08:24 [WARN] drift detected - triggering retrain
03:08:24 [INFO] retrain triggered (window=last_30_days)
03:08:39 [OK] new_nrmse=4.75% - production promoted: SolarCast_Predictor v2
```

장마철은 계획했던 "발전량 절반" 가정 대신 실제 구간을 넣었는데 드리프트로 잡히지 않았습니다. 모델이 3년치 장마를 이미 보고 배웠기 때문이고, HAIC 실습의 "정상 변동성 vs 이상 드리프트" 구분과 같은 논리로 발표에서 쓸 수 있습니다. 학습 범위 밖인 설비 변화(신규 편입)만 드리프트로 잡힙니다.

데모 화면 — 아래 순서가 발표 데모 순서입니다. 로컬에서 `project_solar/README.md` 실행 순서대로 띄우면 같은 화면이 나옵니다 (<http://localhost:8010>).

1. 대시보드 초기 화면: 업로드 → 드리프트 시뮬레이션 → 파이프라인 → 재학습 로그 4단 구성

SolarCast · 태양광

입 발전량 예측 · 드리프트 감지 기반 재학습 파이프라인

MODEL_SOURCE=mlflow 기준

발전 실적 데이터 업로드

data/uploads/ 는 여기서 업로드한 CSV만 쌓이는 곳입니다. 학습(train_and_register.py 등)은 항상 가장 최근에 올라온 파일을 사용합니다. data/sample_samcheonpo2_daily.csv (상천포 2호기 2023-01~2026-08 실적)를 올려보세요 (컬럼: Date, generation_kwh · 최소 35행).

Choose File

No file chosen

업로드

드리프트 시뮬레이션

업로드된 실제 실적에서 35일 구간을 잘라 /predict/batch-test 로 전송합니다 (가짜 데이터를 만들지 않습니다). 정상 = 최근 35일 · 장마철 = 2026-06-20부터 35일 · 신규 발전소 편입 = 최근 35일 × 1.8 (학습 범위 밖 → 드리프트). 드리프트가 감지되면 아래 파이프라인이 감지 → fine-tuning → (nRMSE ≤ 8% 통과 시) 재배포까지 진행됩니다.

정상 배치 (최근 35일)

장마철 배치 (2026-06-20~)

신규 발전소 편입 (×1.8)

드리프트 감지 기반 재학습 파이프라인

아직 실행 안 함

위에서 배치를 보내면, 그 요청이 실제로 거쳐간 단계를 아래에 표시합니다 (자동 스케줄러는 없고, 수동 트리거 방식입니다).

데이터 수집
요청 수신
대기 중

데이터 모니터링
예측 기록 누적
대기 중

드리프트 감지
nRMSE vs 8%
대기 중

재학습 트리거
조건 충족 시 시작
대기 중

모델 학습
fine-tuning
대기 중

모델 등록
MLflow Registry
대기 중

배포
Production 승격
대기 중

재학습 로그

드리프트 감지·재학습 트리거는 logs/aiops.log 에 그대로 기록됩니다. [WARN] drift detected → [INFO] retrain triggered → [OK] new_nrmse=...% promoted 순서로 남는지 아래에서 직접 열어 확인하세요.

aiops.log (506B)

server_day1.log (2046B)

server_day2_day3.log (5420B)

server_day3_shoot.log (6707B)

simulate_drift_day3.log (617B)

train_and_register_day2.log (637B)

train_baseline_v1.log (186B)

SolarCast · 태양광 일 발전량 예측 B2B 서비스 데모 (HAIC 실습 구조 재사용 · serving_app/static/index.html)

2. 정상 배치(최근 35일) 전송: nRMSE 4.4% ≤ 8% → 드리프트 없음, 파이프라인은 감지 단계에서 정상 종료

SolarCast · 태양광

입 발전량 예측 · 드리프트 감지 기반 재학습 파이프라인

MODEL_SOURCE=mlflow 기준

발전 실적 데이터 업로드

data/uploads/ 는 여기서 업로드한 CSV만 쌓이는 곳입니다. 학습(train_and_register.py 등)은 항상 가장 최근에 올라온 파일을 사용합니다. data/sample_samcheonpo2_daily.csv (상천포 2호기 2023-01~2026-08 실적)를 올려보세요 (컬럼: Date, generation_kwh · 최소 35행).

Choose File

No file chosen

업로드

드리프트 시뮬레이션

업로드된 실제 실적에서 35일 구간을 잘라 /predict/batch-test 로 전송합니다 (가짜 데이터를 만들지 않습니다). 정상 = 최근 35일 · 장마철 = 2026-06-20부터 35일 · 신규 발전소 편입 = 최근 35일 × 1.8 (학습 범위 밖 → 드리프트). 드리프트가 감지되면 아래 파이프라인이 감지 → fine-tuning → (nRMSE ≤ 8% 통과 시) 재배포까지 진행됩니다.

정상 배치 (최근 35일)

장마철 배치 (2026-06-20~)

신규 발전소 편입 (×1.8)

● 정상 (드리프트 없음 · nRMSE 4.3% ≤ 8%)

2026-07-28 ~ 2026-08-31 · 클라이언트 계산 nRMSE: 4.30%

```
{
  "predictions": [
    3430.5,
    3547.4,
    3514.3,
    3833,
    3705.2,
    3279.4,
    2251.2,
    2408.2,
    3273.2,
    3545.4,
    3791.9,
    3445.7,
    3684.2,
    3627,
    3724
  ]
}
```

드리프트 감지 기반 재학습 파이프라인

마지막 실행 0초 전

위에서 배치를 보내면, 그 요청이 실제로 거쳐간 단계를 아래에 표시합니다 (자동 스케줄러는 없고, 수동 트리거 방식입니다).

데이터 수집

요청 수신

완료

데이터 모니터링

예측 기록 누적

완료

드리프트 감지

nRMSE vs 8%

완료

재학습 트리거

조건 충족 시 시작

대기 중

모델 학습

fine-tuning

대기 중

모델 등록

MLflow Registry

대기 중

배포

Production 승격

대기 중

재학습 로그

드리프트 감지·재학습 트리거는 logs/aiops.log 에 그대로 기록됩니다. [WARN] drift detected → [INFO] retrain triggered → [OK] new_nrmse=...% promoted 순서로 남는지 아래에서 직접 열어 확인하세요.

aiops.log (506B)

server_day1.log (2046B)

server_day2_day3.log (5420B)

server_day3_shoot.log (6960B)

simulate_drift_day3.log (617B)

train_and_register_day2.log (637B)

train_baseline_v1.log (186B)

SolarCast · 태양광 일 발전량 예측 B2B 서비스 데모 (HAIC 실습 구조 재사용 · serving_app/static/index.html)

3. 신규 발전소 편입 배치(×1.8) 전송: nRMSE 9~10% > 8% → 드리프트 → fine-tuning → 게이트 통과 → 새 버전 승격, aiops.log 3줄

SolarCast · 태양광

일 발전량 예측 · 드리프트 감지 기반 재학습 파이프라인

MODEL_SOURCE=mlflow 기준

발전 실적 데이터 업로드

data/uploads/ 는 여기서 업로드한 CSV만 쌓이는 곳입니다. 학습(train_and_register.py 등)은 항상 가장 최근에 올라온 파일을 사용합니다. data/sample_samcheonpo2_daily.csv (상천포 2호기 2023-01~2026-08 실적)를 올려보세요 (컬럼: Date, generation_kwh · 최소 35행).

Choose File

No file chosen

업로드

드리프트 시뮬레이션

업로드된 실제 실적에서 35일 구간을 잘라 /predict/batch-test 로 전송합니다 (가짜 데이터를 만들지 않습니다). 정상 = 최근 35일 · 장마철 = 2026-06-20부터 35일 · 신규 발전소 편입 = 최근 35일 × 1.8 (학습 범위 밖 → 드리프트). 드리프트가 감지되면 아래 파이프라인이 감지 → fine-tuning → (nRMSE ≤ 8% 통과 시) 재배포까지 진행됩니다.

정상 배치 (최근 35일)

장마철 배치 (2026-06-20~)

신규 발전소 편입 (×1.8)

● 드리프트 nRMSE 9.32% → 재학습 완료 · 새 버전 승격 (new_nrmse=4.64%)

2026-07-28 ~ 2026-08-31 · 클라이언트 계산 nRMSE: 9.32%

```
{
  "predictions": [
    4732.2,
    4857.3,
    4816,
    5273.7,
    5093.2,
    4522.8,
    3174,
    3312.9,
    4418.8,
    4751.1,
    5054.3,
    4663.9,
    5120.4,
    4969,
    5112.2
  ]
}
```

드리프트 감지 기반 재학습 파이프라인

마지막 실행 0초 전

위에서 배치를 보내면, 그 요청이 실제로 거쳐간 단계를 아래에 표시합니다 (자동 스케줄러는 없고, 수동 트리거 방식입니다).

데이터 수집

요청 수신

완료

데이터 모니터링

예측 기록 누적

완료

드리프트 감지

nRMSE vs 8%

감지됨

재학습 트리거

조건 충족 시 시작

완료

모델 학습

fine-tuning

완료

모델 등록

MLflow Registry

완료

배포

Production 승격

완료

재학습 로그

드리프트 감지·재학습 트리거는 logs/aiops.log 에 그대로 기록됩니다. [WARN] drift detected → [INFO] retrain triggered → [OK] new_nrmse=...% promoted 순서로 남는지 아래에서 직접 열어 확인하세요.

aiops.log (759B)

server_day1.log (2046B)

server_day2_day3.log (5420B)

server_day3_shoot.log (8359B)

simulate_drift_day3.log (617B)

train_and_register_day2.log (637B)

train_baseline_v1.log (186B)

```
2026-10-01 03:08:24,331 [WARNING] [WARN] drift detected - triggering retrain
2026-10-01 03:08:24,338 [INFO] [INFO] retrain triggered (window=last_30_days)
2026-10-01 03:08:39,882 [INFO] [OK] new_nrmse=4.75% - production promoted: SolarCast_Predictor v2
2026-10-01 03:12:21,897 [WARNING] [WARN] drift detected - triggering retrain
2026-10-01 03:12:21,904 [INFO] [INFO] retrain triggered (window=last_30_days)
2026-10-01 03:12:40,439 [INFO] [OK] new_nrmse=4.74% - production promoted: SolarCast_Predictor v3
2026-10-01 03:13:21,784 [WARNING] [WARN] drift detected - triggering retrain
2026-10-01 03:13:21,785 [INFO] [INFO] retrain triggered (window=last_30_days)
2026-10-01 03:13:39,259 [INFO] [OK] new_nrmse=4.64% - production promoted: SolarCast_Predictor v4
```

SolarCast · 태양광 일 발전량 예측 B2B 서비스 데모 (HAIC 실습 구조 재사용 · serving_app/static/index.html)

4. Swagger /docs : 엔드포인트 8개, /predict 실행 결과의 model_version 이 재배포 후 버전으로 바뀜

SolarCast - 태양광 일 발전량 예측 Serving & AIOps

0.1.0 OAS 3.1

[/openapi.json](#)

default



POST	/predict	Predict	▼
POST	/predict/batch-test	Batch Test	▼
GET	/health	Health	▼
POST	/data/upload	Upload	▼
GET	/data/window	Window	▼
GET	/data/status	Status	▼
GET	/logs	List Logs	▼
GET	/logs/{filename}	Read Log	▼

Schemas



BatchTestRequest > Expand all object

BatchTestResponse > Expand all object

Body_upload_data_upload_post > Expand all object

DailyPoint > Expand all object

HTTPValidationError > Expand all object

PredictRequest > Expand all object

PredictResponse > Expand all object

ValidationError > Expand all object

POST

/predict

Predict

⌵

받는 것 : {"sequence": [{"generation_kwh": 3320.8}, ... 14개]} 돌려줄 것: {"predicted_kwh": 3410.5, "model_version": "production-v1"}

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

⌵

Edit Value

Schema

```
{
  {
    "generation_kwh": 4187.1
  },
  {
    "generation_kwh": 3991.3
  },
  {
    "generation_kwh": 3863.5
  },
  {
    "generation_kwh": 684.7
  },
  {
    "generation_kwh": 2586.2
  },
  {
    "generation_kwh": 2947.6
  }
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://localhost:8010/predict' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "sequence": [
    {
      "generation_kwh": 836.4
    },
    {
      "generation_kwh": 3894.7
    },
    {
      "generation_kwh": 3921.1
    },
    {
      "generation_kwh": 4248
    },
    {
      "generation_kwh": 3659.9
    },
    {
      "generation_kwh": 4035.2
    },
    {
      "generation_kwh": 4896.3
    }
  ]
}'
```

Request URL

http://localhost:8010/predict

Server response

Code

Details

200

Response body

```
{
  "predicted_kwh": 3181.2,
  "model_version": "production-v3"
}
```

⌵

Download

Response headers

```
content-length: 56
content-type: application/json
date: Thu, 01 Oct 2026 03:13:15 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

⌵

Controls Accept header.

Example Value

Schema

Page 13 of 14

```

{
  "predicted_kwh": 0,
  "model_version": "string"
}

```

422 Validation Error No links

Media type

application/json

Example Value | Schema

```

{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}

```

증빙 원본은 저장소 `team_solar/snapshots/` (로그 6개, 캐치 5장)에 있습니다.

기대 효과와 리스크

발표의 한 줄: "예측 오차가 8%를 넘어가는 순간을 당일 감지해 재학습하므로, 중개사업자는 정산금을 놓치는 날을 수주에서 하루로 줄인다." 정산금 단가를 확인하면 발전소 용량 × 일수로 금액으로 환산해 넣으세요.

리스크	영향	대응
공개 데이터에 기상 컴럼이 없거나 날짜가 안 맞음	피쳐 확장 불가	발전량 단일 피썰(HAIC와 동일 구조)로 먼저 끝까지 돌리고, 기상은 있으면 추가
발전량 0인 밤 시간·비 오는 날이 많아 MAPE가 폭발	게이트가 항상 탈락	지표를 설비용량 대비 nRMSE로 고정 (위 운영 설계에 반영)
드리트 시뮬레이션이 정상 배치에도 오탐	데모 실패	실습처럼 랜덤워크가 아니라 실제 데이터 마지막 35일을 정상 배치로, 거기에 일사량·발전량을 절반으로 줄인 것을 드리트 배치로
발표 시간 부족 (Day3 오후 한 번에 통합)	끝까지 안 돌아감	1일 발전량 · 피쳐 2개(발전량, 일사량)로 범위를 익히고, 시간별 예측·KS 검정은 "확장 과제"로만 언급